Reproduction and Extensions of a PINN for European and American Option Pricing

Zhixuan Chen Grace Pan

1 Introduction

Physics-Informed Neural Networks (PINNs) incorporate the underlying physics into the loss function, making them a useful framework for solving differential equations. The governing structure is represented by the differential equation together with its initial conditions (ICs) and boundary conditions (BCs). In the option pricing paper of Socaciu and Paşcu (2025), PINNs are proposed as a method for solving the Black–Scholes (BS) PDE for financial derivatives. In this project, we reproduced the Black–Scholes European call experiment in PyTorch and compared the PINN result against a standard MLP and the closed-form BS solution when applicable. We then tested five extensions, including an additional boundary condition, residual-based adaptive refinement (RAR), different activation functions, a put–call parity check, and an experiment on an option type for which the classical model does not have a closed-form solution.

2 Reproduction of Results

2.1 Black–Scholes PDE

Options are financial contracts whose values depend on an underlying asset. A European option can only be exercised at maturity, while an American option can be exercised at any time before maturity. Call options give the right to buy the asset, and put options give the right to sell it.

For a European call option with value $u(S,t)$, where S is the underlying asset price, K is the strike price, T is the maturity time, σ is the volatility, and r is the risk-free interest rate, the BS PDE is

$$u_t + \frac{1}{2}\sigma^2 S^2 u_{SS} + rSu_S - ru = 0.$$

At maturity, the holder will only exercise the option if doing so gives a profit, that is, if the asset price exceeds the strike price: $u(S,T) = \max(S-K, 0)$.

The lower boundary condition for a call is $u(0,t) = 0$. A zero asset price gives no reason to pay a positive strike price to buy it, therefore the call value must also be zero.

For large S, the option behaves approximately like holding the asset and paying the strike at maturity, giving

$$u(S,t) \approx S - Ke^{-r(T-t)}.$$

This asymptotic relation is not explicitly enforced in the paper's Black–Scholes example, but it motivates our first extension.

On the other hand, although the BS model is the classical model for option pricing, it does not have a closed-form solution for the American put. Unlike its European counterpart, an American option can be exercised at any time before maturity. This leads to a free-boundary problem. One

formulation is the variational inequality

$$\max\left(u_t + \frac{1}{2}\sigma^2 S^2 u_{SS} + rSu_S - ru,\ K - S - u\right) = 0.$$

Hence, in our final extension, we consider how a PINN can be adapted to this American-option setting by modifying the payoff, the boundary conditions, and the loss.

2.2 PINNs architecture and loss

The paper proposed that instead of solving the BS PDE by a traditional discretization method, PINNs can be trained to satisfy the PDE inside the domain, the initial and terminal condition, the boundary conditions, and optionally the observed market or numerical data.

Total loss. The total loss for a PINN is

$$\mathcal{L} = a \cdot \mathcal{L}_{\text{PDE}} + b \cdot \mathcal{L}_{\text{BCs}} + c \cdot \mathcal{L}_{\text{ICs}} + d \cdot \mathcal{L}_{data}$$

$\mathcal{L}_{\text{PDE}}$ denotes the PDE loss, $\mathcal{L}_{\text{BCs}}$ denotes the Boundary Conditions (BC) loss, $\mathcal{L}_{\text{ICs}}$ denotes the Initial Conditions (ICs) loss, and $\mathcal{L}_{\text{data}}$ denotes the data loss. The authors note that the data loss is optional. The input is (S, t), where S is the asset price and t is time. The output is the approximated option value $\hat{u}(x, t; \theta)$. The parameters θ are chosen to minimize the total loss.

PDE loss. We define the residual of the Black–Scholes PDE as

$$f(S, t) = u_t + \frac{1}{2}\sigma^2 S^2 u_{SS} + rSu_S - ru.$$

The PDE loss enforces that this residual is close to zero at collocation points:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_f}\sum_{i=1}^{N_f} |f(S_i, t_i)|^2 .$$

Initial conditions (ICs) loss. At maturity $t = T$, the option must match its payoff:

$$u(S, T) = \max(S - K, 0).$$

This is enforced by

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_{ic}}\sum_{i=1}^{N_{ic}} |u_\theta(S_i, T) - \max(S_i - K, 0)|^2 .$$

Boundary condition (BC) loss. For a European call, the lower boundary is

$$u(0, t) = 0.$$

This leads to the boundary loss

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_{bc}}\sum_{i=1}^{N_{bc}} |u_\theta(0, t_i)|^2 .$$

2.3 Analysis of Reproduced Results

In this project, we do not include a data loss term. Since the paper treats it as optional, we focus on solving the BS equation using the PDE together with terminal and boundary constraints.

We re-implemented the model from scratch in PyTorch. The network is a 3-layer MLP with architecture $\mathbb{R}^2 \to 20 \to 20 \to \mathbb{R}^1$ and tanh activation, trained with Adam (learning rate 10^{-3}) for 20,000 epochs. At each step, $N_f = 1{,}000$ collocation points are sampled uniformly from $[0, S_{\max}] \times [t_0, T]$; all loss terms carry equal weight. We set $\Delta t = \frac{1}{128}$ for the lower-BC grid (versus $\frac{1}{52}$ in the paper) to better stabilize the $S \approx 0$ region.

The results show that the PINN outperforms the MLP especially in the $S \approx K$ region. This demonstrates that enforcing the PDE constraints is crucial for accurately approximating the solution. This also highlights the advantage of PINNs over purely data-driven approaches when the underlying physical structure is known.

3 Extensions

3.1 Right boundary condition

In the original paper, the authors did not explicitly include the right boundary condition. However, as seen in our implementation results, the approximation error increases in this region. Therefore, we introduce an additional boundary condition at $S = S_{\max}$:

$$u(S_{\max}, t) \approx S_{\max} - Ke^{-r(T-t)}.$$

This boundary condition is motivated by the asymptotic behavior of the BS model for large S, as discussed in Section 2.1. It gives the additional loss:

$$\mathcal{L}_{\text{BC}}^{\text{hi}} = \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} \left| u_\theta(S_{\max}, t_i) - \left(S_{\max} - Ke^{-r(T-t_i)} \right) \right|^2.$$

As shown in Figure 1, the baseline PINN deviates from the reference solution for large S. Incorporating the right BC aligns the solution with the expected closed-form solution and significantly improving accuracy in this region.

3.2 Residual-based Adaptive Refinement (RAR)

The BS solution varies most rapidly near $S \approx K$, where the PDE residual tends to be largest. With uniform random sampling, collocation points are spread evenly across the domain, so many points end up in smooth regions where they contribute little to training. RAR improves on this by periodically identifying where the residual is large and moving points from low-error regions to high-error ones, so the network focuses on where it is needed most. Figure 2 shows that the dominant error in the baseline model occurs in the large-S region. However, the additional right BC gives a larger improvement than RAR.

3.3 Swish Activation Function and the Comparisons of Activation Functions

Since the BS model requires computing u_{SS}, ReLU is not suitable because its second derivative is zero almost everywhere. We experiment with alternative activation functions, specifically tanh and the Swish activation function: $Swish(x) = x\sigma(x)$. The latter has a non-saturating gradient

for large inputs, which would help suppress error growth in the large S region.

More specifically, we observe that with the right boundary condition, tanh produces more stable and accurate results. With the right boundary condition, tanh produces more stable and accurate results. The MAE with tanh is 0.0986, compared with 0.1391 for Swish, and the relative L^2 error with tanh is 0.0062, compared with 0.0107 for Swish.

3.4 Put–Call Parity Check

Put–Call parity is a classical relation between a European put and a European call:

$$C(S,t) - P(S,t) = S - Ke^{-r(T-t)}.$$

Accurate call and put models would yield this equation. As shown in Figure 3, the learned solutions closely match the theoretical relation across the domain. This indicates that the PINN preserves the structural consistency implied by the governing equation.

3.5 American Put Option

We follow the formulation in [2], Section 3.2.2, and model the American put option. The loss is augmented with two additional terms: a complementarity loss and an early-exercise penalty term. The first enforces the product condition between the BS operator and the exercise gap, while the second enforces $V \geq \max(K - S, 0)$. The resulting surface (Figure 4) exhibits the expected qualitative behavior: high values for small S, decreasing as S increases, and approaching zero for large S. This indicates that the PINN captures the structure of the free-boundary problem.

4 Final Model Selection

We find that the addition of the right boundary condition and the usage of tanh add the most value and give the best performance. We note that RAR is not included in the final model, as it mainly improves computational efficiency but does not improve accuracy.

5 Conclusion and Future Work

In this project, we reproduced the results in the paper and explored five extensions. The results demonstrate that PINNs can effectively approximate the BS PDE. For future work, we can consider:

- incorporating real market data into the loss function as data loss,
- extending to more complex option types, such as barrier options,
- extending to high-dimensional(multi-asset) options, where traditional methods become computationally infeasible but PINNs scale well.

References

[1] T. Socaciu and P. Paşcu, *Physics-Informed Neural Networks in Pricing Financial Options*, BRAIN. Broad Research in Artificial Intelligence and Neuroscience, vol. 16, no. 2, pp. 474–488, 2025.

[2] A. Dhiman and P. Hu, *Physics-Informed Neural Networks for Option Pricing*, arXiv preprint arXiv:2312.06711, 2023.

Figures

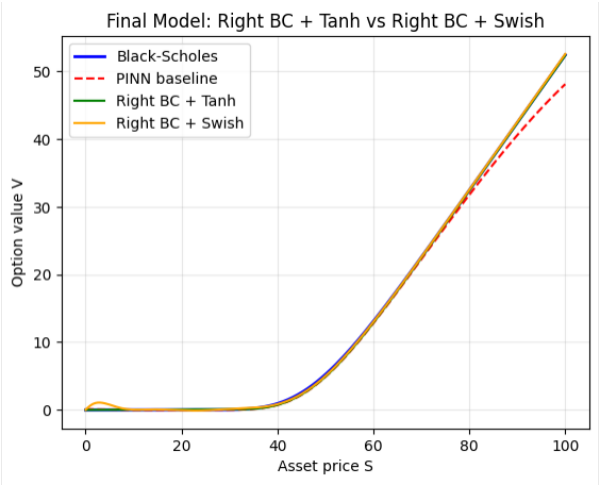

Figure 1: Comparison of baseline and final models.

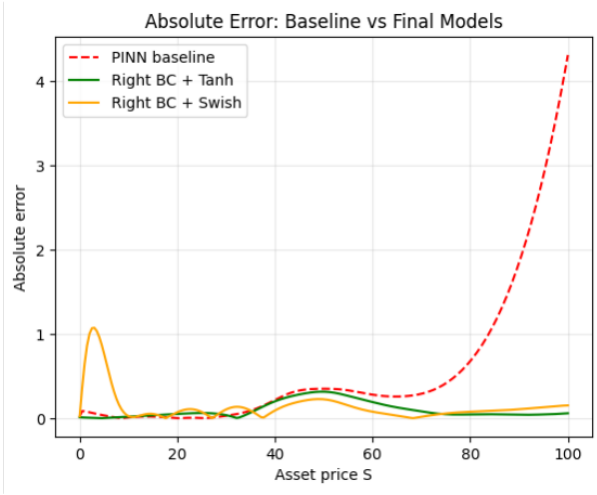

Figure 2: Absolute error comparison between the baseline and final models.

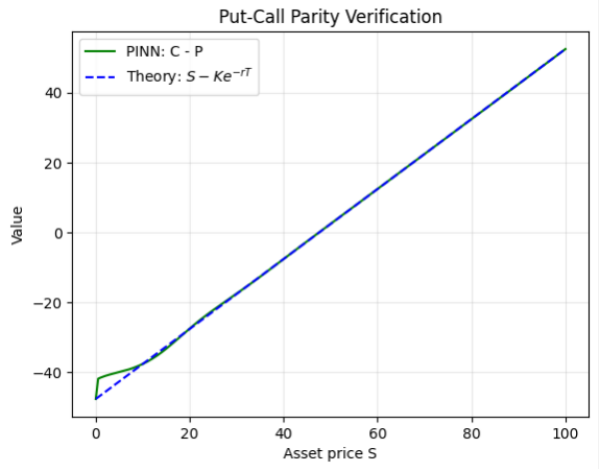

Figure 3: Put–call parity verification.

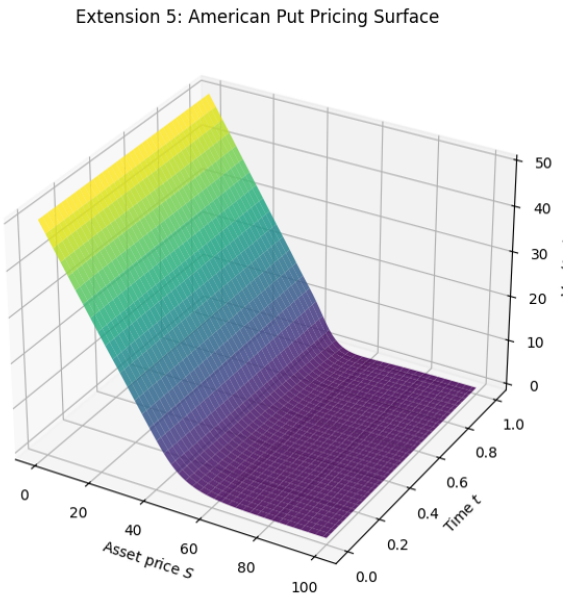

Figure 4: American put pricing surface.